



Hybrid Recommendation System Lab

Brief Overview

This note covering **Hybrid Recommendation Systems** was created from the [2.1.6. Lab Solution: Building a Neural Network Hybrid Recommendation System](#) YouTube video.

It walks through data preprocessing, Apache Beam pipeline creation, TensorFlow model building, training & evaluation, and cloud deployment for a hybrid recommendation system.

Key Points

- How to combine collaborative-filtering embeddings with content metadata.
 - Building and running an Apache Beam pipeline to generate sharded CSVs for TensorFlow.
 - Designing feature columns, model architecture, and loss functions in TensorFlow 2.
 - Deploying and hyper-parameter tuning the model on Cloud ML Engine.
-



Data Preprocessing & Hybrid Dataset Construction

Hybrid Recommendation System – a model that merges *collaborative-filtering embeddings* with *content-based attributes* (e.g., title, author, category) to predict the next item a user will engage with.

- **Environment setup**
 - Switch kernel to **Python 2** for Apache Beam (current limitation).
 - Uninstall and reinstall the cloud-dataflow package.
 - Reset notebook kernel after changes.
- **BigQuery steps**
 1. Set project ID and compute region as environment variables.
 2. Create an **ML dataset** that joins:
 - User embeddings (10 latent factors).
 - Item embeddings (10 latent factors).
 - Content metadata: visitor_id, content_id, category, title, author, months_since_epoch.
- **Label definition**

The **label** for the supervised neural network is the *next content ID* the user actually consumed.

- **Feature list**

- visitor_id (int)
- content_id (current) (int)
- category (string)
- title (string)
- author (string, may be null)
- months_since_epoch (int) – days from **Jan 1 1970**.
- user_factor_1 ... user_factor_10 (float)
- item_factor_1 ... item_factor_10 (float)



Apache Beam Pipeline (Python 2)

- **Goal:** Transform the BigQuery result into sharded CSV files ready for TensorFlow.
- **CSV schema:**

Column	Description
next_content_id (label)	Target ID to predict
visitor_id	User identifier
content_id	Currently viewed content
category	Content category
title	Content title (UTF-8)
author	Author name (nullable)
months_since_epoch	Temporal feature
user_factor_1 ... user_factor_10	User latent vectors
item_factor_1 ... item_factor_10	Item latent vectors

- **Encoding:** Use **UTF-8** for German characters; cast everything else to string to avoid errors.
- **Runner selection**

- DirectRunner → local test mode (adds a LIMIT to the query).
 - DataflowRunner → full-scale cloud execution, output written to **Google Cloud Storage**.
 - **Train / Eval split**
 - Compute `hash_id = ABS(FARM_FINGERPRINT(visitor_id))`.
 - `bucket = hash_id % 10`.
 - Buckets **0-8** → training (90 %).
 - Bucket **9** → evaluation (10 %).
 - *Custom splits (e.g., 80/20) can be implemented by changing the modulus logic.*
-

Vocabulary & Statistics Generation

- **Purpose:** Provide lookup tables for categorical features (category, author, content_id).
- **Process**
 1. Run a separate Beam job that **GROUP BY** each target column to collect unique values.
 2. Write each vocabulary to a text file (vocab_category.txt, vocab_author.txt, vocab_content_id.txt).
 3. Compute **counts** and **means** (e.g., average months_since_epoch).
- **Resulting statistics**

Feature	Unique values	Example entries
author	1,103	"Munson", "Epic", ...
category	3	"Lifestyle", "Stars", "Culture News"
content_id	15,634	12345, 67890, ...
months_since_epoch (mean)	–	573.6

These numbers are later used as **default values** (e.g., mean months) when a record is missing.

Model Building (TensorFlow 2 + TF-Hub)

Feature Column – an abstraction that tells TensorFlow how to interpret raw data (categorical, numeric, text) and how to transform it for a DNN.

1. Global Variables

- Load vocabulary counts & means from the files generated above.
- Define CSV column order (non-factor + factor columns).

2. Default Values

Column	Default (if missing)
String features (category, title, author)	"unknown"
months_since_epoch	573 (mean)
Embedding factors	0.0 (float)

3. Input Function (CSV decoding)

- Reads CSV, applies defaults, builds a **features dictionary** and a **single-label tensor** (next_content_id).
- Supports **training** (shuffle, repeat) and **evaluation** (no shuffle, single epoch).

4. Feature Column Definitions

Feature	Column type	Transformation
content_id	categorical_column_with_hash_bucket	Hash bucket → embedding (dimension configurable).
category	categorical_column (vocab file)	indicator_column → dense one-hot.
title	text_embedding_column (TF-Hub)	German pre-trained embeddings (e.g., 50-dim). Fixed (non-trainable) by default.
author	categorical_column_with_hash_bucket	Hash bucket → embedding_column (configurable dim).
months_since_epoch	numeric_column → bucketized_column	Buckets: 0-100, 100-200, ..., 600-700.

Crossed feature	<code>crossed_column</code> (category × months bucket)	<code>indicator_column</code> → captures interaction effects.
User & item factors	<code>numeric_column</code> (float)	Directly fed as dense inputs.

5. Model Architecture

1. **Input layer** – receives the concatenated dense tensor from all feature columns.
2. **Hidden layers** – defined by a list of unit sizes passed via command line (e.g., [128, 64]).
 - Use **ReLU** (or other non-linearities).
3. **Output layer** – dense layer with **units = # of content IDs** (no activation, produces **logits**).

6. Loss & Optimization

- **Loss**: `sparse_softmax_cross_entropy` between logits and label indices.
- **Metrics**: accuracy, top-k recall (implicit in evaluation).

7. Prediction Logic

- Apply **softmax** to logits → normalized probabilities (sum = 1).
- `argmax` of logits → predicted class index.
- Map index → actual `content_id` name using the **content ID vocabulary lookup table**.
- Return a dictionary:
 - `class_ids` (int)
 - `class_names` (string)
 - `probabilities` (float vector)
 - `logits` (raw scores)



Training, Evaluation & Hyper-Parameter Tuning

- **Local training**: Run the model in the notebook to verify correctness of feature pipelines and architecture.
- **Cloud training**: Submit the same estimator to **AI Platform** (or equivalent) for scalable training.

- **Hyper-parameter tuning**: Vary embedding dimensions, hidden layer sizes, learning rate, etc., to find the best configuration before launching a large-scale job.
-

Model Function & Mode Handling

Sparse label – an integer index representing the *next content ID*; it is exclusive (either the current or the next item, never a mix), allowing the use of sparse tensors for loss computation.

- **Mode branching**
 - **PREDICT** – returns an EstimatorSpec with only predictions (no loss, no training).
 - **EVAL** – returns an EstimatorSpec containing:
 - loss (computed on the evaluation batch)
 - eval_metric_ops (accuracy & top-K accuracy)
 - No export outputs (early return).
 - **TRAIN** – asserts that the mode is indeed TRAIN before proceeding, preventing accidental weight updates in other modes.
 - **Optimizer selection**
 - Default: **Adam** (works well for deep neural nets).
 - Alternative optimizers (e.g., **Adagrad**, **SGD**) can be swapped by changing the optimizer variable.
 - Learning rate is supplied via the **hyper-parameter dictionary**.
 - **Training step**
 - `train_op = optimizer.minimize(loss, global_step=tf.train.get_global_step())`
 - Returns an EstimatorSpec with loss and train_op; no predictions or eval metrics are attached during training.
-

Metrics & Evaluation

Metric	Description	Usage in EVAL mode
Accuracy	Fraction of exact matches between predicted and true next_content_id.	<code>tf.metrics.accuracy(labels, predictions)</code>

Top-K Accuracy	Proportion of instances where the true label appears in the top K predicted scores (commonly $K = 10$).	<code>tf.metrics.mean(tf.nn.in_top_labels, K))</code>
Custom Scalars	User-defined scalar values (e.g., loss, learning-rate) that can be logged for monitoring.	Added to <code>tf.summary.scalar</code> calls within the model function.

Top-K accuracy – a metric that evaluates how often the correct item is among the K highest-scoring predictions, useful for recommendation scenarios where multiple suggestions are presented.

Training Procedure & Loop

1. **Set logging** – `tf.logging.set_verbosity(tf.logging.INFO)` to expose step-wise information.
2. **Create estimator** –

```
estimator = tf.estimator.Estimator(
    model_fn=custom_model_fn,
    model_dir=output_dir,
    params={
        "feature_columns": feature_columns,
        "hidden_units": hidden_units,
        "num_classes": num_content_ids,
        "learning_rate": lr,
        "top_k": top_k,
        "bucket": gcs_bucket
    })
```

3. **Define specs**
 - **TrainingSpec** – supplies the training input function, `train_steps` (from CLI).

- **EvalSpec** – supplies the evaluation input function; steps=None to run through the entire eval file (single epoch).
4. **Exporter** – tf.estimator.LatestExporter writes a SavedModel checkpoint after each evaluation cycle.
 5. **Run loop** – tf.estimator.train_and_evaluate(estimator, train_spec, eval_spec).
-

Serving Input Function (Cloud ML Engine)

- **Placeholder creation** – builds a feature_placeholders dict keyed by column name.
 - **Non-factor columns** (strings): skip index0 (next_content_id) and index -1 (months_since_epoch), which is a float.
 - Add a special placeholder **months_since_epoch** as tf.float32.
- **Factor columns** – all latent factors are tf.float32; they are stacked into a single vector per example.
- **Shape handling** – each placeholder has shape [None] (batch dimension) for scalars, or [None, factor_dim] for vector factors.
- **Return** – tf.estimator.export.ServingInputReceiver(features, receiver_tensors).

Serving input function – a routine that maps raw request data to the model's expected feature dictionary, enabling deployment on Cloud ML Engine.

Local Execution Workflow

Step	Action
1 Output directory	Choose a fresh model_dir; delete any existing tree to avoid collisions.
2 Arguments dict	Populate with command-line parameters (feature columns, hidden units, etc.) for local testing.
3 Call training	Invoke the train_and_evaluate function; monitor console logs for progress.

4 Dependency check	Ensure tensorflow_hub is installed (pip install tensorflow_hub) to avoid import errors.
5 Verify	After training, confirm that a SavedModel is written to model_dir.

Cloud ML Engine Execution & Hyper-Parameter Tuning

Job Submission

- **Required flags:** --region, --module-name, --package-path, --runtime-version, plus all hyper-parameters.
- **Output location** – a GCS bucket (e.g., gs://my-bucket/training/).

Hyper-Parameter Tuning (YAML)

```
trainingInput:
  scaleTier: BASIC_GPU
  hyperparameters:
    goal: MAXIMIZE
    hyperparameterMetricTag: accuracy
    maxTrials: 20
    maxParallelTrials: 5
    enableTrialEarlyStopping: true
    params:
      - parameterName: learning_rate
        type: DOUBLE
        minValue: 0.0001
        maxValue: 0.01
        scaleType: LOG_LINEAR
      - parameterName: batch_size
        type: INTEGER
        minValue: 64
        maxValue: 512
        scaleType: UNIT_LINEAR_SCALE
      - parameterName: hidden_units
        type: CATEGORICAL
        categoricalValues: ["64,32", "128,64", "256,128"]
      - parameterName: content_embedding_dim
        type: INTEGER
```

```
minValue: 8
maxValue: 32
- parameterName: author_embedding_dim
  type: INTEGER
  minValue: 4
  maxValue: 16
```

- **Metrics to optimize** – typically **accuracy** or **top-K accuracy**.
- **Trial configuration** – each trial runs a full training job with a distinct hyper-parameter set; the best trial's parameters are later used for the final large-scale model.

Post-Tuning Execution

1. Retrieve the optimal hyper-parameters from the tuning job's summary.
2. Launch a **full training job** with a larger train_steps count, using the discovered settings.
3. Export the final SavedModel for serving.



End-to-End Hybrid Recommendation Workflow

Component	Role
Collaborative-filtering embeddings	Capture user-item interaction patterns (latent factors).
Content-based attributes (title, author, category)	Provide semantic context for items lacking interaction data.
Custom TensorFlow estimator	Merges embeddings and content features, computes loss, and outputs predictions.
Metrics (accuracy, top-K)	Evaluate recommendation quality during development.
Serving input function	Enables deployment on Cloud ML Engine, handling both string and float features.
Hyper-parameter tuning	Systematically explores model capacities (embedding dims, hidden layers) to maximize predictive performance.

Cloud training	Scales computation across many workers for large datasets.
-----------------------	--

Hybrid recommendation system – the integrated model described above leverages both collaborative-filtering latent vectors and rich content metadata, delivering superior predictions compared to using either source alone.